

DIGITAL FORENSICS PROJECT REPORT

File Integrity Verification using Hashing Techniques

Prepared By: Sanjana S

Tools Used: Kali Linux, Python, SQLite3, FTK Imager

1. Introduction

Digital forensics plays a critical role in modern cybersecurity investigations. One of its core objectives is to ensure the authenticity and integrity of digital evidence. This project demonstrates the process of verifying file integrity using cryptographic hash algorithms such as MD5 and SHA256. By computing these hashes for selected files, investigators can confirm that the files have not been altered or tampered with during the course of an investigation.

2. Objective

The objective of this project is to design and implement a Python-based digital forensics tool that scans files, calculates their hash values (MD5 and SHA256), and stores them securely in a database. This allows for the verification of file integrity over time by comparing current hashes with the originally computed values.

3. Tools and Environment

- Operating System: Kali Linux 2024
- Programming Language: Python 3.12
- Database: SQLite3
- Supporting Tools: FTK Imager, Hashlib Library, CSV/Excel for reporting
- IDE/Editor: Visual Studio Code / Terminal

4. Methodology

The methodology followed in this project involved several structured steps, from setup to data reporting. Each phase ensures the forensic soundness of the process and reliable results.

1. Step 1: Created a working directory named ForensicsFileAnalyzer.
2. Step 2: Added a subfolder 'test_files' containing sample evidence files (file1.txt, file2.txt).

3. Step 3: Developed a Python script (file_analyzer.py) that computes MD5 and SHA256 hashes for each file.
4. Step 4: Designed a SQLite3 database (forensics.db) to store all file details and hash values.
5. Step 5: Executed the script to analyze the files and insert their metadata and hash results into the database.
6. Step 6: Exported the database records into a CSV file (forensic_report.csv).
7. Step 7: Formatted and visualized the data in Excel for clear representation and saved as a final forensic report (PDF).

5. Implementation Details

The project was implemented using Python on Kali Linux. The key modules used include hashlib for hashing, sqlite3 for database management, and os for file handling.

Below are the main scripts developed for this project.

The Python script utilizes the hashlib module to compute the MD5 and SHA256 hash values. Each file's metadata—name, path, size, and analysis timestamp—is also recorded. The results are stored in a SQLite database for easy retrieval and cross-verification. This database can later be queried to check for any discrepancies or tampering.

- Key Python libraries used: hashlib, os, sqlite3, datetime

File: create_db.py

Purpose: Initializes the SQLite database and creates the required table for forensic analysis results.

```
import sqlite3
```

```
# Create database and table
```

```
conn = sqlite3.connect('forensics.db')
```

```
cursor = conn.cursor()
```

```
cursor.execute("""
```

```

CREATE TABLE IF NOT EXISTS forensic_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    file_name TEXT,
    file_path TEXT,
    md5_hash TEXT,
    sha256_hash TEXT,
    file_size INTEGER,
    analysis_time TEXT
)
"""
conn.commit()
conn.close()

```

File: file_analyzer.py

Purpose: Calculates file hashes (MD5 and SHA256), collects metadata, and stores results in the database.

```

import hashlib, os, sqlite3, datetime

def hash_file(file_path):
    md5 = hashlib.md5()
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        data = f.read()
        md5.update(data)
        sha256.update(data)
    return md5.hexdigest(), sha256.hexdigest()

conn = sqlite3.connect('forensics.db')
cursor = conn.cursor()

folder = "test_files"
for file in os.listdir(folder):
    path = os.path.join(folder, file)
    size = os.path.getsize(path)

```

```

md5, sha256 = hash_file(path)
time = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
cursor.execute('INSERT INTO forensic_data (file_name, file_path,
md5_hash, sha256_hash, file_size, analysis_time) VALUES (?, ?, ?, ?, ?, ?)',
               (file, path, md5, sha256, size, time))
conn.commit()
conn.close()

```

```

sanjana@ADMIN: ~/forensic
(Message from Kali developers)
This is a minimal installation of Kali Linux, you likely
want to install supplementary tools. Learn how:
= https://www.kali.org/docs/troubleshooting/common-minimum-setup/
(Run: "touch ~/.hashlogin" to hide this message)
sanjana@ADMIN: ~
$ mkdir ForensicsFileAnalyzer
cd ForensicsFileAnalyzer
mkdir: cannot create directory 'ForensicsFileAnalyzer': File exists

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ nano create_db.py

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ python3 create_db.py
Database and table created successfully.

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ ls
create_db.py  forensics.db

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ nano file_analyzer.py

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ python3 file_analyzer.py
Enter folder path to analyze:
X Folder not found.

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ ls ~/Desktop
ls: cannot access '/home/sanjana/Desktop': No such file or directory

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ ls -
ForensicsFileAnalyzer

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ mkdir ~/ForensicsFileAnalyzer/test_files

```

```
sanjana@ADMIN: ~/Forensics x + ~
GNU nano 8.6 create_db.py
import sqlite3

# Connect or create the DB
conn = sqlite3.connect('forensics.db')
cursor = conn.cursor()

# Create table
cursor.execute('''
CREATE TABLE IF NOT EXISTS file_analysis (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    file_name TEXT,
    file_path TEXT,
    md5_hash TEXT,
    sha256_hash TEXT,
    file_size INTEGER,
    analysis_time TEXT
)
''')

conn.commit()
conn.close()

print("✅ Database and table created successfully.")

Write to File: create_db.py
^G Help      M-D DOS Format  M-A Append     M-B Backup File  T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

```
sanjana@ADMIN: ~/Forensics x + ~
~$ ls -
ForensicsFileAnalyzer
[sanjana@ADMIN] ~/ForensicsFileAnalyzer
$ mkdir ~/ForensicsFileAnalyzer/test_files
[sanjana@ADMIN] ~/ForensicsFileAnalyzer
$ echo "This is a test file 1" > ~/ForensicsFileAnalyzer/test_files/file1.txt
echo "Digital forensics test file 2" > ~/ForensicsFileAnalyzer/test_files/file2.txt
[sanjana@ADMIN] ~/ForensicsFileAnalyzer
$ python3 file_analyzer.py
Enter folder path to analyze: /home/sanjana/ForensicsFileAnalyzer/test_files

File analyzed: file2.txt
MD5: 916725656313e29fcbeddef24f4f282f
SHA256: 8bbac61b105090f5d7dbec398d3ae12fb253ce11bef84d1e73c0c93a48b383bc8
Size: 38 bytes
Logged at: 2025-10-27 17:24:10

File analyzed: file1.txt
MD5: 5d5776f4ac159bbd38954fa98d79fcb9
SHA256: 9acd6f784b420d8cd5f1a61aa6cc8583abf69dd2513bb0e7cee6d624bb466b6d
Size: 22 bytes
Logged at: 2025-10-27 17:24:10

✅ Analysis completed and saved to forensics.db
[sanjana@ADMIN] ~/ForensicsFileAnalyzer
$ sqlite3 forensics.db
Command 'sqlite3' not found, but can be installed with:
sudo apt install sqlite3

[sanjana@ADMIN] ~/ForensicsFileAnalyzer
$ sudo apt update
sudo apt install sqlite3 -y
[sudo] password for sanjana:
Hit:1 http://http.kali.org/kali kali-last-snapshot InRelease
All packages are up to date.
Installing:
sqlite3
```

```

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ sudo apt update
sudo apt install sqlite3 -y
[sudo] password for sanjana:
Hit:1 http://http.kali.org/kali kali-last-snapshot InRelease
All packages are up to date.
Installing:
  sqlite3

Suggested packages:
  sqlite3-doc

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 384 kB
  Space needed: 681 kB / 1,825 GB available

Get:1 http://kali.download/kali kali-last-snapshot/main amd64 sqlite3 amd64 3.46.1-7 [384 kB]
Fetched 384 kB in 2s (197 kB/s)
Selecting previously unselected package sqlite3.
(Reading database ... 31895 files and directories currently installed.)
Preparing to unpack .../sqlite3_3.46.1-7_amd64.deb ...
Unpacking sqlite3 (3.46.1-7) ...
Setting up sqlite3 (3.46.1-7) ...

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ sqlite3 --version
3.46.1 2024-08-13 09:16:08 c9c2ab54baf5f46368f1b4f35d849cd3f880e6fc2b6c60e91b16c63f69aalt1 (64-bit)

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ sqlite3 forensics.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables;
Error: unknown command or invalid arguments: ".tables;". Enter ".help" for help
sqlite> .tables
file_analysis
sqlite> .schema files;
sqlite> SELECT * FROM files;
Parse error: no such table: files
sqlite> .mode column

```

```

Space needed: 681 kB / 1,825 GB available

Get:1 http://kali.download/kali kali-last-snapshot/main amd64 sqlite3 amd64 3.46.1-7 [384 kB]
Fetched 384 kB in 2s (197 kB/s)
Selecting previously unselected package sqlite3.
(Reading database ... 31895 files and directories currently installed.)
Preparing to unpack .../sqlite3_3.46.1-7_amd64.deb ...
Unpacking sqlite3 (3.46.1-7) ...
Setting up sqlite3 (3.46.1-7) ...

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ sqlite3 --version
3.46.1 2024-08-13 09:16:08 c9c2ab54baf5f46368f1b4f35d849cd3f880e6fc2b6c60e91b16c63f69aalt1 (64-bit)

sanjana@ADMIN: ~/ForensicsFileAnalyzer
$ sqlite3 forensics.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables;
Error: unknown command or invalid arguments: ".tables;". Enter ".help" for help
sqlite> .tables
file_analysis
sqlite> .schema files;
sqlite> SELECT * FROM files;
Parse error: no such table: files
sqlite> .mode column
.headers on
SELECT * FROM file_analysis;
extra argument: 'on'
sqlite> .mode column
.headers on
SELECT * FROM file_analysis;
extra argument: 'on'
sqlite> .mode column
.headers on
extra argument: 'on'
sqlite> SELECT * FROM file_analysis;
1|file2.txt|/home/sanjana/ForensicsFileAnalyzer/test_files/file2.txt|916725656313e29fcbdd5f24f4282f|8bbac61b105b90fd7d9e398d1ae12fb253ce11be784d1e73c6c93
a48b383bc8|38|2025-10-27 17:24:18
2|file1.txt|/home/sanjana/ForensicsFileAnalyzer/test_files/file1.txt|5d5776f4ac159bbd38954fa98d79fcb9|9acd6f784b420d8cd5f1a61aa6cc8583abf69dd2513bb0e7cee6d6
20b466b6d|22|2025-10-27 17:24:18

```

```
sanjana@ADMIN ~/Forensics$ ...> Ctrl + D
...>

Parse error: near "sqlite": syntax error
sqlite> .headers ON .exit sqlite> .exit .quit sqlite> Ctrl + C sqlite> .exit C
*--- error here

sanjana@ADMIN:~/ForensicsFileAnalyzer$
$ sqlite3 forensics.db
.mode column
.headers on
SELECT * FROM file_analysis;
SQLite version 3.46.1 2024-08-13 09:16:00
Enter ".help" for usage hints.
sqlite> .mode column
.headers on
SELECT * FROM file_analysis;
extra argument: "on"
sqlite> .mode column
sqlite> .headers on
sqlite> SELECT * FROM file_analysis;
id  file_name  file_path  analysis_time  md5_hash  sha256_hash
-----
1  file2.txt  /home/sanjana/ForensicsFileAnalyzer/test_files/file2.txt  916725656313e29fcbddcf24f4f282f  8bbac61b105b98f5d7d8e198d3ae12fb253ce11bef80d1e73
c8c93a48b38  30  2025-10-27 17:24:10  3bc8
2  file1.txt  /home/sanjana/ForensicsFileAnalyzer/test_files/file1.txt  5d5779f4ac159bbd38954fa98d79fcb9  9acd6f789b428d8cd5f1a61aa6cc8583abf69dd2513bb0e7c
ee6d6248b46  22  2025-10-27 17:24:10  6b6d

sqlite> .mode csv
.headers on
.output forensic_report.csv
SELECT * FROM file_analysis;
.output stdout
extra argument: "on"
sqlite>
```

```
sanjana@ADMIN ~/Forensics$ ...> Ctrl + D
...>

Parse error: near "sqlite": syntax error
sqlite> .headers ON .exit sqlite> .exit .quit sqlite> Ctrl + C sqlite> .exit C
*--- error here

sanjana@ADMIN:~/ForensicsFileAnalyzer$
$ sqlite3 forensics.db
.mode column
.headers on
SELECT * FROM file_analysis;
SQLite version 3.46.1 2024-08-13 09:16:00
Enter ".help" for usage hints.
sqlite> .mode column
.headers on
SELECT * FROM file_analysis;
extra argument: "on"
sqlite> .mode column
sqlite> .headers on
sqlite> SELECT * FROM file_analysis;
id  file_name  file_path  analysis_time  md5_hash  sha256_hash
-----
1  file2.txt  /home/sanjana/ForensicsFileAnalyzer/test_files/file2.txt  916725656313e29fcbddcf24f4f282f  8bbac61b105b98f5d7d8e198d3ae12fb253ce11bef80d1e73
c8c93a48b38  30  2025-10-27 17:24:10  3bc8
2  file1.txt  /home/sanjana/ForensicsFileAnalyzer/test_files/file1.txt  5d5779f4ac159bbd38954fa98d79fcb9  9acd6f789b428d8cd5f1a61aa6cc8583abf69dd2513bb0e7c
ee6d6248b46  22  2025-10-27 17:24:10  6b6d

sqlite> .mode csv
.headers on
.output forensic_report.csv
SELECT * FROM file_analysis;
.output stdout
extra argument: "on"
sqlite> .mode csv
sqlite> .headers on
sqlite> .output forensic_report.csv
sqlite> SELECT * FROM file_analysis;
sqlite> .output stdout
...> SELECT * FROM file_analysis;
Parse error: near "sqlite": syntax error
sqlite> .mode csv sqlite> .headers on sqlite> .output forensic_report.csv sqli
*--- error here

sqlite> .mode csv
.headers on
.output forensic_report.csv
SELECT * FROM file_analysis;
.output stdout
.exit
extra argument: "on"
sqlite> .headers on
sqlite> .mode csv
.headers on
.output forensic_report.csv
SELECT * FROM file_analysis;
.output stdout
.exit
extra argument: "on"
sqlite> .mode csv
.headers on
.output forensic_report.csv
...
extra argument: "on"
sqlite> .mode csv
sqlite> .headers on
sqlite> .output forensic_report.csv
sqlite> SELECT * FROM file_analysis;
sqlite> .output stdout
```



```
Setting up libmailtools-perl (2.22-1) ...
Setting up librt64:amd64 (1:1.2.1-1.2+b2) ...
Setting up libhttp-daemon-perl (6.16-1) ...
Setting up cpp (4:14.2.0-1) ...
Setting up mesa-libgl1:amd64 (25.0.7-2) ...
Setting up libxmu6:amd64 (2:1.1.3-3+b8) ...
Setting up libghel1:amd64 (25.0.7-2) ...
Setting up libgl1-mesa-dri:amd64 (25.0.7-2) ...
Setting up libxaw7:amd64 (2:1.8.16-1) ...
Setting up x11-xserver-utils (9.7+11) ...
Setting up libegl-mesa0:amd64 (25.0.7-2) ...
Setting up libegl1:amd64 (1.7.0-1+b2) ...
Setting up libglx-mesa0:amd64 (25.0.7-2) ...
Setting up libglx0:amd64 (1.7.0-1+b2) ...
Setting up libgl1:amd64 (1.7.0-1+b2) ...
Setting up rutils (0.16.3-20250305+dfsg1-1) ...
update-alternatives: using /usr/bin/rutils to provide /usr/bin/x-terminal-emulator (x-terminal-emulator) in auto mode
Setting up x11-utils (7.7+9) ...
Setting up liblwp-protocol-https-perl (6.14-1) ...
Setting up libwww-perl (6.70-1) ...
Setting up libxml-parser-perl (2.47-1+b3) ...
Setting up libxml-twig-perl (1:3.54-1) ...
Setting up libnet-dbus-perl (1.2.0-2+b1) ...
Processing triggers for libc-bin (2.41-12) ...

(sanjana@ADMIN) ~/ForensicsFileAnalyzer
$ xdg-open forensic_report.csv

(sanjana@ADMIN) ~/ForensicsFileAnalyzer
$
```

6. Findings

The analysis confirmed that each file generated a unique hash signature. The MD5 and SHA256 values act as digital fingerprints, uniquely identifying each file. If a file were altered in any way, even by a single bit, the computed hash would differ dramatically—indicating tampering.

Below is a sample representation of the results obtained during the file analysis phase:

ID	File Name	File Path	MD5 Hash	SHA256 Hash	Size (Bytes)
1	file1.txt	/home/sanjana/ForensicsFileAnalyzer/test_files/file1.txt	5d5776f4ac159bbd38954fa98d79fcb9	9acd6f784b420d8cd5f1a61aa6cc8503abf69dd2513bb0e7cee6d624bb466b6d	22

2	file2.txt	/home/s anjana/F orensics FileAnal yzer/test _files/fil e2.txt	9167256563 13e29fcbedd ef24f4f282f	8bbac61b105b9 0f5d7dbe398d3a e12fb253ce11be f84d1e73c0c93a 48b303bc8	30
---	-----------	--	--	--	----

7. Conclusion

This project effectively demonstrates the process of digital evidence integrity verification using Python and database management. By integrating cryptographic hashing algorithms (MD5, SHA256) with database storage and structured reporting, the project provides a practical foundation for real-world digital forensics operations.

The results validate that cryptographic hash functions are reliable tools for identifying unauthorized data modification, ensuring the chain of custody, and maintaining the forensic soundness of evidence.

8. Future Scope

The project can be extended with additional functionalities, such as:

- Integration with FTK or Autopsy for advanced evidence management.
- GUI interface for file selection and live hash comparison.
- Support for bulk file scanning and report generation in HTML or PDF.
- Integration with cloud storage for remote forensic validation.